

CE 791 HPC - Final Exam

Andrew Navratil

Problem 3: Simpson's Rule Integration

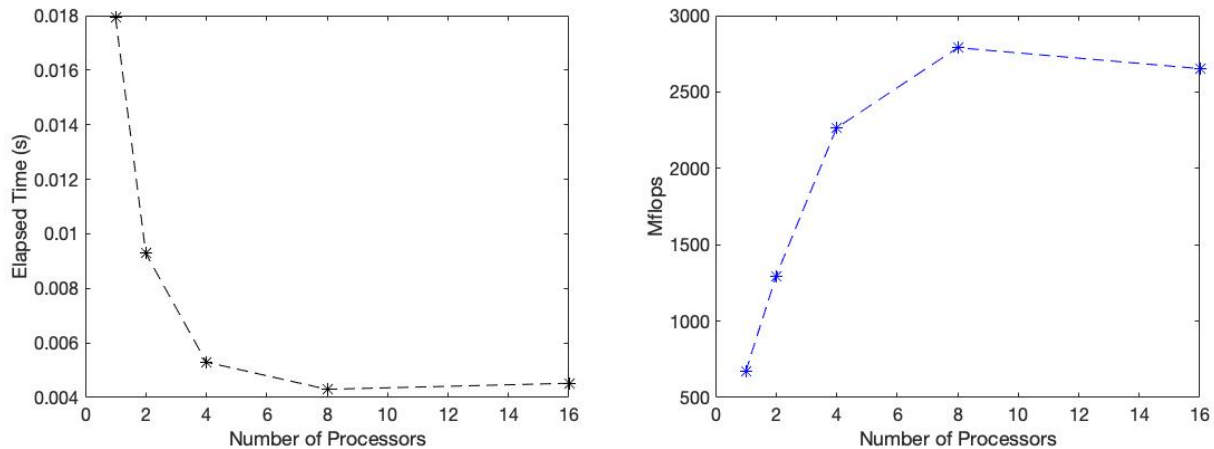


Fig. 1 Execution Times, Mflops for Problem 3

Benchmarking plots show an increase in mflops and decrease in execution time for this parallel implementation up through the use of 8 processors, however using 16 showed a decrease from 8. Explanation for this decrease is unknown, perhaps it is due to the use of 2 MPI_REDUCE calls and that causes too much parallel communication overhead at 16 procs.

Problem 4: Sum 1-p

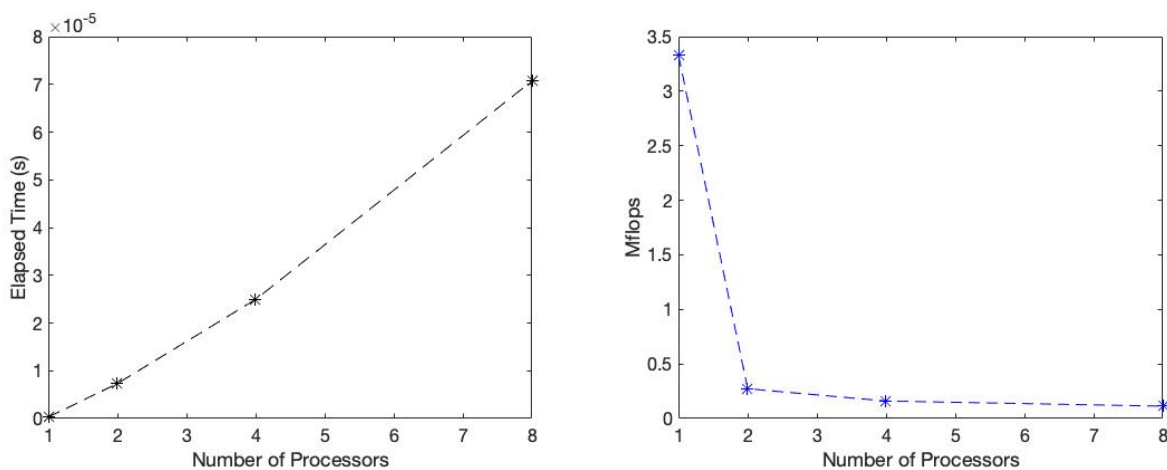


Fig. 2 Execution Times, Mflops for Problem 4

Testing the parallel sum code, summing 1 to num procs p, shows a decrease in Mflops and increase in execution times as num procs increases. This is likely due to the simplicity of the simple single addition/assignment for each

processor, so parallel communication overhead is the biggest issue for parallel scalability on this problem.

Problem 5: 6-digit conditional combinations

Parallel code was written for a specific number of processors for this problem, that being 9, where each processor performed calculations for a combination with the starting digit being the processor number plus one. The total number of combinations was calculated as 527787.

Problem 7: 1D GW Flow with v2 Jacobi, RBGS, OpenMP

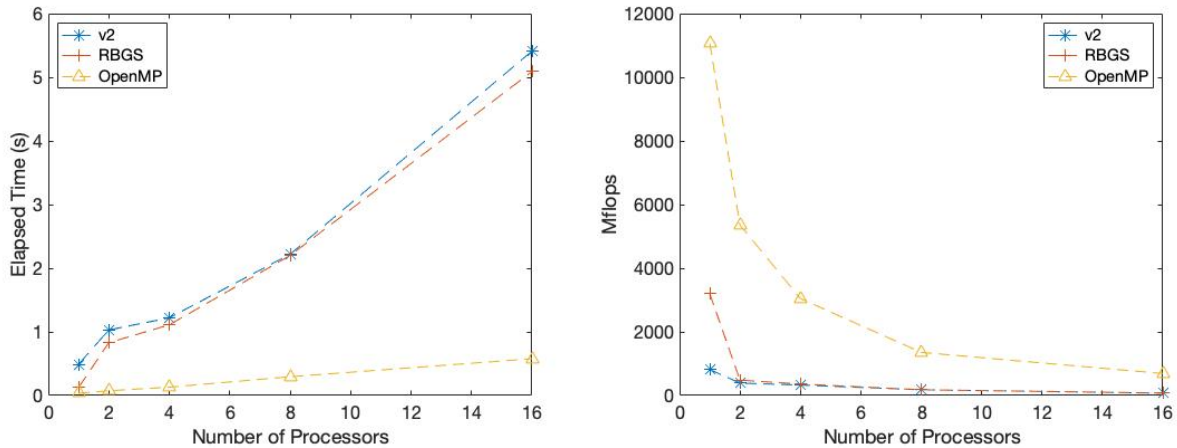


Fig. 3 Execution Times, Mflops for Problem 7

Benchmarking plots again show an increase in mflops and decrease in execution time for this parallel implementation up through the use of all 16 processors. The cause for drop in performance is still unknown. The OpenMP version achieved significantly better performance than MPI Red-Black Gauss-Seidel or MPI Jacobi v2. The accuracy (solution error) for Jacobi was 0.0101 and for both the MPI and OpenMP versions of RBGS it was 0.0196.

Problem 9: Containment Source

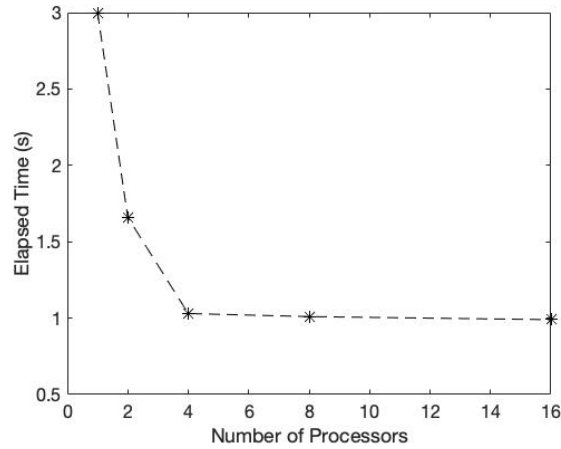


Fig. 4 Execution Times for Problem 9

Parallel random search was implemented for combinations of x_0, y_0 and M_0 , using integer values in the respective ranges. `MPI_MINLOC` was utilized with `MPI_ALLREDUCE` in order to get the processor finding the minimum least squares difference between the observed and calculated values. `MPI_GATHER` was then used to get the actual values of $\min x_0, y_0, M_0$. Note that observation values in the pdf are assumed to have been printed like a typical x-y grid with origin in bottom left, so top y row is 400m and bottom y row is 0m.

Benchmarking the execution times shows the expected decrease as number of processors increases, with the largest drop seen up to 4, with slight improvements using 8 or 16.

The optimized combinations were different on each run, presumably due to the amount of possible combinations there are (1e6 number of random combinations was tried on each run). Runs were also executed with 1e8 trials, and still produced different combinations. The values received for each final run of num procs at 1e6 trials is:

Num Procs	x0	y0	M0
1	836	299	36
2	791	203	77
4	335	194	21
8	342	204	74
16	589	343	35

Fortran Code: Problem 3

```
1  ! Andrew Navratil
2  ! CE 791 Final Exam – Problem 3 Simpson Rule Integration
3
4  program p3
5  implicit none
6  include "mpif.h"
7
8  ! === constants ===
9  real(8), parameter :: PI = 4.D0*DATAN(1.D0)
10
11  integer, parameter :: a = 0.0, b = 1.0
12
13  integer, parameter :: n = 1e6 ! num subintervals
14  integer, parameter :: numtrials = 1024 ! num trials for benchmarking
15
16  ! === variables ===
17  real(8) mysum1, mysum2, totalsum1, totalsum2, h, fint, x
18  integer i,j,k,numops
19
20  character(len=50) :: filename
21
22  ! === MPI variables ===
23  real(8) time1,time2,avgtime,mflops,ierr
24  integer nprocs,iam
25
26  !=====
27
28  ! initialize MPI
29  call MPI_INIT(ierr)
30  call MPI_COMM_RANK(MPI_COMM_WORLD, iam, ierr)
31  call MPI_COMM_SIZE(MPI_COMM_WORLD, nprocs, ierr)
32
33  ! start timer for mflops calculation
34  time1 = MPI_Wtime()
35
36  ! calculate interval size
37  h = 1.0d0/n
38
39  ! outer loop to run numtrials for benchmarking
40  do j = 1,numtrials
41      mysum1 = 0.d0
42      mysum2 = 0.d0
43
44      ! calculate sums, distribute workload among all procs
45      do i = iam+1,n,nprocs
46          x = h*i
47          if (mod(i,2).eq.0) then
48              mysum2 = mysum2 + f(x) ! evens
49          else
50              mysum1 = mysum1 + f(x) ! odds
51          end if
```

```

52     end do
53
54     ! collect all the partial sums (root proc is 0)
55     call MPI_REDUCE(mysum1,totalsum1,1,MPI_DOUBLE_PRECISION, &
56         MPI_SUM,0,MPI_COMM_WORLD,ierr)
57     call MPI_REDUCE(mysum2,totalsum2,1,MPI_DOUBLE_PRECISION, &
58         MPI_SUM,0,MPI_COMM_WORLD,ierr)
59
60     ! perform the simpson's rule integration version
61     fint = (dble(b-a)/dble(3.*n))
62     fint = fint * (f(dble(a))+4.*totalsum1+2.*totalsum2+f(dble(b)))
63 end do
64
65 ! end timer
66 time2 = MPI_Wtime()
67 avgtime = (time2-time1)/numtrials
68
69 ! print mflops and execution time to final.out file
70 if (iam.eq.0) then
71     write(*,*) 'value of integral is ',fint,' error to pi is ',abs(fint-PI)
72     numops = 12 ! +,-,/,*,*,3+,2* in loop, and /,*,+ in f
73     mflops = numops*n*1e-6/avgtime
74
75     write(*,*) 'nprocs =',nprocs
76     write(*,*) 'mflops =',mflops
77     write(*,*) 'avg time (per trial) =',avgtime
78 end if
79
80 ! finalize MPI
81 call MPI_FINALIZE(ierr)
82
83 contains
84
85 !=====
86 function f(x)
87     real(8) x,f
88
89     f = 4.d0/(1.d0 + x*x)
90
91 end function f
92
93 !=====
94 end program p3

```

Fortran Code: Problem 4

```
1  ! Andrew Navratil
2  ! CE 791 Final Exam – Problem 4 Sum
3
4  program p4
5  implicit none
6  include "mpif.h"
7
8      ! === constants ===
9      integer, parameter :: numtrials = 1024  ! num trials for benchmarking
10
11     ! === variables ===
12     integer x,totalsum,sumcheck
13     integer i,j,k,numops
14
15     ! === MPI variables ===
16     real(8) time1,time2,avgtime,mflops,ierr
17     integer nprocs,iam
18
19     !=====
20
21     ! initialize MPI
22     call MPI_INIT(ierr)
23     call MPI_COMM_RANK(MPI_COMM_WORLD, iam, ierr)
24     call MPI_COMM_SIZE(MPI_COMM_WORLD, nprocs, ierr)
25
26     ! start timer for mflops calculation
27     time1 = MPI_Wtime()
28
29     ! outer loop to run numtrials for benchmarking
30     do j = 1,numtrials
31         x = iam+1
32
33         call MPI_REDUCE(x,totalsum,1,MPI_INT, &
34             MPI_SUM,0,MPI_COMM_WORLD,ierr)
35     end do
36
37     ! end timer
38     time2 = MPI_Wtime()
39     avgtime = (time2-time1)/numtrials
40
41     ! print mflops and execution time to final.out file
42     if (iam.eq.0) then
43         sumcheck = nprocs*(nprocs+1)/2
44         write(*,*) 'reduction sum = ',totalsum
45         write(*,*) 'sum check on 0 = ',sumcheck
46         numops = 1
47         mflops = numops*nprocs*1e-6/avgtime
48
49         write(*,*) 'nprocs =',nprocs
50         write(*,*) 'mflops =',mflops
51         write(*,*) 'avg time (per trial) =',avgtime
```

```
52     end if
53
54     ! finalize MPI
55     call MPI_FINALIZE(ierr)
56
57 end program p4
```

Fortran Code: Problem 5

```
1  ! Andrew Navratil
2  ! CE 791 Final Exam – Problem 5 6-digit combinations
3
4  program p5
5  implicit none
6  include "mpif.h"
7
8      ! === constants ===
9
10     ! === variables ===
11     integer mycount, totalcount
12     integer d1,d2,d3,d4,d5,d6,dsum
13     logical docount
14
15     ! === MPI variables ===
16     real(8) time1,time2,avgtime,ierr
17     integer nprocs,iam
18
19     !=====
20
21     ! initialize MPI
22     call MPI_INIT(ierr)
23     call MPI_COMM_RANK(MPI_COMM_WORLD, iam, ierr)
24     call MPI_COMM_SIZE(MPI_COMM_WORLD, nprocs, ierr)
25
26     ! start timer for mflops calculation
27     time1 = MPI_Wtime()
28
29     ! loop for each of the 6 digits, note 1st one can't be 0
30     mycount = 0.d0
31     !do d1 = 1,9
32     d1 = iam+1 ! written specifically for 9 procs
33     do d2 = 0,9
34     do d3 = 0,9
35     do d4 = 0,9
36     do d5 = 0,9
37     do d6 = 0,9
38         ! the 6 digit number is d1d2d3d4d5d6
39         docount = .true.
40         dsum = 0.d0
41
42         ! check if any two consecutive digits are the same
43         if (d1.eq.d2.or.d2.eq.d3.or.d3.eq.d4.or.d4.eq.d5.or.d5.eq.d6) then
44             docount = .false.
45         end if
46
47         ! check if sum of digits is 7,11,13
48         dsum = d1+d2+d3+d4+d5+d6
49         if (dsum.eq.7.or.dsum.eq.11.or.dsum.eq.13) then
50             docount = .false.
51         end if
```



```

52
53     if (docount) then
54         mycount = mycount + 1
55     end if
56 end do
57 end do
58 end do
59 end do
60 end do
61 !end do
62
63 ! aggregate counts
64 call MPI_REDUCE(mycount,totalcount,1,MPI_INT,MPI_SUM,0,MPI_COMM_WORLD,ierr)
65
66 ! print mflops and execution time to final.out file
67 if (iam.eq.0) then
68     write(*,*) 'num combinations = ',totalcount
69
70     write(*,*) 'nprocs =',nprocs
71     write(*,*) 'avg time (per trial) =',avgtime
72 end if
73
74 ! finalize MPI
75 call MPI_FINALIZE(ierr)
76
77 end program p5

```

Fortran Code: Problem 7 v2

```
1  !=====
2  ! jacobi for 1-D Laplacian
3  !=====
4  program jacobi
5  !=====
6  include "mpif.h"
7  !=====
8
9  !=====
10 ! initialize main parameters
11 !=====
12 Integer, Parameter :: size=1000, nit = 5e4
13 !real(8), Parameter :: h0 = 1.d0, hL = 0.1d0 ! version 0
14 real(8), Parameter :: h0 = 1.0, hL = 0.0 ! version 1 BCs
15 real(8), Parameter :: x0 = 0.0, xL = 10.0 ! version 1 x domain (0-10m)
16 real(8), Parameter :: dx = (xL-x0)/size ! compute grid spacing / delta x
17 real(8), parameter :: a = 0.007, b = -0.07, c = 0.2 ! k consts
18 real(8), parameter :: c0 = -0.0055, c1 = 0.5 ! analytical sln consts
19 real(8), parameter :: qarea = 100 ! avg cross sectional area
20 integer, parameter :: ndims = 1 ! version 4 topology funcs
21
22 !=====
23 ! declare variables
24 !=====
25 real(8), allocatable :: h(:), hnew(:), k(:)
26 real(8) time1, time2, mflops, errsqr, error, sumsqerr
27 integer nprocs,i,iam,n,left,right,j
28 integer count,tag1,tag2
29 integer status(MPI_STATUS_SIZE)
30 integer commId
31 integer status_array(MPI_STATUS_SIZE, 4), req(4)
32 real(8) :: xval, fanal, Q, dhdx
33 integer :: dims(ndims)
34 logical :: isoperiodic(ndims), reorder
35 character(len=50) :: filename
36
37 !=====
38 ! initialize MPI
39 !=====
40 call MPI_INIT(ierr)
41 call MPI_COMM_RANK(MPI_COMM_WORLD, iam, ierr)
42 call MPI_COMM_SIZE(MPI_COMM_WORLD, nprocs, ierr)
43
44 !=====
45 ! assume size is perfectly divisible by nprocs
46 !=====
47 n = size/nprocs
48
49 !=====
50 ! allocate arrays
51 !=====
```

```

52     allocate (h(0:n+1),hnew(0:n+1),k(0:n+1))
53
54     !=====
55     ! start timer
56     !=====
57     time1=MPI_Wtime()
58
59     !=====
60     ! initialize message passing parameters
61     !=====
62     count=1
63     tag1=1
64     tag2=2
65     !left=iam-1
66     !right=iam+1
67     if (iam.eq.0) then
68         left = MPI_PROC_null
69     else
70         left = iam-1
71     end if
72     if (iam.eq.nprocs-1) then
73         right = MPI_PROC_NULL
74     else
75         right = iam+1
76     end if
77
78     !=====
79     !initalize h and k values
80     !=====
81     if (iam.eq.0) then
82         h(0) = h0
83     end if
84     if (iam.eq.nprocs-1) then
85         h(n+1) = hL
86     end if
87
88     xval = (iam*n) * dx
89     k(0) = a*xval**2 + b*xval + c
90     xval = ((iam+1)*n + 1) * dx
91     k(n+1) = a*xval**2 + b*xval + c
92
93     do i=1,n
94         h(i) = 0.d0
95
96         xval = (iam*n + i) * dx
97         k(i) = a*xval**2 + b*xval + c
98     enddo
99
100    !=====
101    !start iterations
102    !=====
103    do j=1,nit
104        !=====
105        ! version 2

```

```

106      !=====
107      ! send to right neighbor and receive from left neighbor
108      call MPI_Sendrecv(h(n), count, MPI_DOUBLE_PRECISION, right, tag1, &
109          h(0), count, MPI_DOUBLE_PRECISION, left, tag1, &
110          MPI_COMM_WORLD, status, ierr)
111      ! send to left neighbor and receive from right neighbor
112      call MPI_Sendrecv(h(1), count, MPI_DOUBLE_PRECISION, left, tag2, &
113          h(n+1), count, MPI_DOUBLE_PRECISION, right, tag2, &
114          MPI_COMM_WORLD, status, ierr)
115
116      ! version 0,1,2,4,5 computation of interior (all but ver 3)
117      !calculate new values of h
118      do i=1,n
119          !hnew(i)=(h(i-1)+h(i+1))/2.d0 ! version 0
120          hnew(i) = ((k(i+1)+k(i))*h(i+1) + (k(i)+k(i-1))*h(i-1)) / &
121              (k(i+1) + 2*k(i) + k(i-1))
122      enddo
123      !update old values
124      do i=1,n
125          h(i)=hnew(i)
126      enddo
127      !reinitialize boundary conditions
128      if (iam.eq.0) h(0)=h0
129      if (iam.eq.nprocs-1) h(n+1)=hL
130
131      ! average leakage flux calculation (only do this one time)
132      ! approximate dh/dx using 2nd order central diff
133      !Q = 0
134      !do i = 1,n
135          !!dhdx = 0.5*(h(i+1) - h(i-1))
136          !dhdx = (h(i) - h(i-1))
137          !Q = Q + k(i) * qarea * dhdx * dx
138      !enddo
139      enddo
140      !=====
141      !end iterations
142      !=====
143
144      !=====
145      !end timer
146      !=====
147      time2=MPI_Wtime()
148      total_time=time2-time1
149
150      !=====
151      ! calculate error using analytical solution
152      !=====
153      errsq=0.d0
154      do i=1,n
155          !hanal=h0+(hL-h0)*(iam*n+i)/(size+1) ! version 0
156
157          xval = (iam*n + i) * dx
158          fanal = (2/sqrt(4*a*c-b*b))*atan((2*a*xval+b)/sqrt(4*a*c-b*b))
159          hanal = c0*fanal + c1

```

```

160         errsq = errsq + (h(i)-hanal)**2
161         !print *, "errsq = ",errsq
162     enddo
163     call MPI_REDUCE(errsq,sumsqerr,1,MPI_DOUBLE_PRECISION,MPI_SUM,0, &
164                  MPI_COMM_WORLD,ierr)
165
166
167     !=====
168     ! print results
169     !=====
170
171     ! write out solution to file for processing in matlab
172     write(filename,"(A8,I1,A4)" 'slndata_',iam,'.txt'
173     open(1,file = filename, status='replace')
174     do i = 1,n
175         write(1,"(I5,2E20.8)" i,k(i),h(i)
176     end do
177     close(1)
178
179     if (iam == 0) then
180         !print *, "sumsqerr = ",sumsqerr
181         error=sqrt(sumsqerr)/dble(size) ! can only do this on proc 0
182
183         !mflops = 2*dble(nit)*dble(size)*1e-6/(total_time) ! version 0
184         mflops = 8*dble(nit)*dble(size)*1e-6/(total_time) ! versions 1-5
185
186         print *, "nprocs =",nprocs
187         print *, "error =",error
188         print *, 'mflops =',mflops
189         print *, 'Total time =',total_time
190
191         !print *, 'Q =',Q
192     endif
193     call MPI_FINALIZE(ierr)
194 end

```

Fortran Code: Problem 7 RBGS

```
1  !=====
2  ! jacobi for 1-D Laplacian
3  !=====
4  program jacobi
5  !=====
6  include "mpif.h"
7  !=====
8
9  !=====
10 ! initialize main parameters
11 !=====
12 Integer, Parameter :: size=1000, nit = 5e4
13 !real(8), Parameter :: h0 = 1.d0, hL = 0.1d0 ! version 0
14 real(8), Parameter :: h0 = 1.0, hL = 0.0 ! version 1 BCs
15 real(8), Parameter :: x0 = 0.0, xL = 10.0 ! version 1 x domain (0-10m)
16 real(8), Parameter :: dx = (xL-x0)/size ! compute grid spacing / delta x
17 real(8), parameter :: a = 0.007, b = -0.07, c = 0.2 ! k consts
18 real(8), parameter :: c0 = -0.0055, c1 = 0.5 ! analytical sln consts
19 real(8), parameter :: qarea = 100 ! avg cross sectional area
20 integer, parameter :: ndims = 1 ! version 4 topology funcs
21
22 !=====
23 ! declare variables
24 !=====
25 real(8), allocatable :: h(:), hnew(:), k(:)
26 real(8) time1, time2, mflops, errsqr, error, sumsqerr
27 integer nprocs,i,iam,n,left,right,j,m
28 integer count,tag1,tag2
29 integer status(MPI_STATUS_SIZE)
30 integer commId
31 integer status_array(MPI_STATUS_SIZE, 4), req(4)
32 real(8) :: xval, fanal, Q, dhdx
33 integer :: dims(ndims)
34 logical :: isoperiodic(ndims), reorder
35 character(len=50) :: filename
36
37 !=====
38 ! initialize MPI
39 !=====
40 call MPI_INIT(ierr)
41 call MPI_COMM_RANK(MPI_COMM_WORLD, iam, ierr)
42 call MPI_COMM_SIZE(MPI_COMM_WORLD, nprocs, ierr)
43
44 !=====
45 ! assume size is perfectly divisible by nprocs
46 !=====
47 n = size/nprocs
48
49 !=====
50 ! allocate arrays
51 !=====
```

```

52     allocate (h(0:n+1),hnew(0:n+1),k(0:n+1))
53
54     !=====
55     ! start timer
56     !=====
57     time1=MPI_Wtime()
58
59     !=====
60     ! initialize message passing parameters
61     !=====
62     count=1
63     tag1=1
64     tag2=2
65     !left=iam-1
66     !right=iam+1
67     if (iam.eq.0) then
68         left = MPI_PROC_null
69     else
70         left = iam-1
71     end if
72     if (iam.eq.nprocs-1) then
73         right = MPI_PROC_NULL
74     else
75         right = iam+1
76     end if
77
78     !=====
79     !initialize h and k values
80     !=====
81     if (iam.eq.0) then
82         h(0) = h0
83     end if
84     if (iam.eq.nprocs-1) then
85         h(n+1) = hL
86     end if
87
88     xval = (iam*n) * dx
89     k(0) = a*xval**2 + b*xval + c
90     xval = ((iam+1)*n + 1) * dx
91     k(n+1) = a*xval**2 + b*xval + c
92
93     do i=1,n
94         h(i) = 0.d0
95
96         xval = (iam*n + i) * dx
97         k(i) = a*xval**2 + b*xval + c
98     enddo
99
100    !=====
101    !start iterations
102    !=====
103    do j=1,nit
104        ! send to right neighbor and receive from left neighbor
105        call MPI_Sendrecv(h(n), count, MPI_DOUBLE_PRECISION, right, tag1, &

```

```

106         h(0), count, MPI_DOUBLE_PRECISION, left, tag1, &
107         MPI_COMM_WORLD, status, ierr)
108     ! send to left neighbor and receive from right neighbor
109     call MPI_Sendrecv(h(1), count, MPI_DOUBLE_PRECISION, left, tag2, &
110         h(n+1), count, MPI_DOUBLE_PRECISION, right, tag2, &
111         MPI_COMM_WORLD, status, ierr)
112
113     ! red-black Gauss-Seidel (0=red,1=black)
114     ! with GS, don't need hnew, just updating h directly
115     do m=0,1
116         do i = m,n,2
117             !h(i)=(h(i-1)+h(i+1))/2.d0
118             h(i) = ((k(i+1)+k(i))*h(i+1) + (k(i)+k(i-1))*h(i-1)) / &
119                 (k(i+1) + 2*k(i) + k(i-1)))
120         enddo
121     end do
122
123     !reinitialize boundary conditions
124     if (iam.eq.0) h(0)=h0
125     if (iam.eq.nprocs-1) h(n+1)=hL
126
127     ! average leakage flux calculation (only do this one time)
128     ! approximate dh/dx using 2nd order central diff
129     !Q = 0
130     !do i = 1,n
131         !!dhdx = 0.5*(h(i+1) - h(i-1))
132         !dhdx = (h(i) - h(i-1))
133         !Q = Q + k(i) * qarea * dhdx * dx
134     !enddo
135 enddo
136 !=====
137 !end iterations
138 !=====
139
140 !=====
141 !end timer
142 !=====
143     time2=MPI_Wtime()
144     total_time=time2-time1
145
146 !=====
147 ! calculate error using analytical solution
148 !=====
149     errsq=0.d0
150     do i=1,n
151         !hanal=h0+(hL-h0)*(iam*n+i)/(size+1) ! version 0
152
153         xval = (iam*n + i) * dx
154         fanal = (2/sqrt(4*a*c-b*b))*atan((2*a*xval+b)/sqrt(4*a*c-b*b))
155         hanal = c0*fanal + c1
156
157         errsq = errsq + (h(i)-hanal)**2
158         !print *, "errsq = ",errsq
159     enddo

```



```

160     call MPI_REDUCE(errsq,sumsqerr,1,MPI_DOUBLE_PRECISION,MPI_SUM,0, &
161                   MPI_COMM_WORLD,ierr)
162
163     !=====
164     ! print results
165     !=====
166     if (iam == 0) then
167         !print *, "sumsqerr = ",sumsqerr
168         error=sqrt(sumsqerr)/dble(size) ! can only do this on proc 0
169
170         !mflops = 2*dble(nit)*dble(size)*1e-6/(total_time) ! version 0
171         mflops = 8*dble(nit)*dble(size)*1e-6/(total_time) ! versions 1-5
172
173         print *, "nprocs =",nprocs
174         print *, "error =",error
175         print *, 'mflops =',mflops
176         print *, 'Total time =',total_time
177
178         !print *, 'Q =',Q
179     endif
180     call MPI_FINALIZE(ierr)
181 end

```

Fortran Code: Problem 7 OpenMP

```
1  !=====
2  ! jacobi for 1-D Laplacian
3  !=====
4  program jacobi
5  !=====
6
7  !=====
8  ! initialize main parameters
9  !=====
10 Integer, Parameter :: size=1000, nit = 5e4
11 !real(8), Parameter :: h0 = 1.d0, hL = 0.1d0 ! version 0
12 real(8), Parameter :: h0 = 1.0, hL = 0.0 ! version 1 BCs
13 real(8), Parameter :: x0 = 0.0, xL = 10.0 ! version 1 x domain (0-10m)
14 real(8), Parameter :: dx = (xL-x0)/size ! compute grid spacing / delta x
15 real(8), parameter :: a = 0.007, b = -0.07, c = 0.2 ! k consts
16 real(8), parameter :: c0 = -0.0055, c1 = 0.5 ! analytical sln consts
17 real(8), parameter :: qarea = 100 ! avg cross sectional area
18 integer, parameter :: ndims = 1 ! version 4 topology funcs
19
20 !=====
21 ! declare variables
22 !=====
23 real(8), allocatable :: h(:), hnew(:), k(:)
24 real(8) time1, time2, mflops, errsqr, error, sumsqr
25 integer nprocs,i,iam,n,left,right,j,m
26 real(8) :: xval, fanal, Q, dhdx
27 integer :: dims(ndims)
28 logical :: isoperiodic(ndims), reorder
29 character(len=50) :: filename
30
31 !=====
32 ! assume size is perfectly divisible by nprocs
33 !=====
34 n = size
35
36 !=====
37 ! allocate arrays
38 !=====
39 allocate (h(0:n+1),hnew(0:n+1),k(0:n+1))
40
41 !=====
42 ! start timer
43 !=====
44 call cpu_time(time1)
45
46 !=====
47 !initialize h and k values
48 !=====
49 if (iam.eq.0) then
50     h(0) = h0
51 end if
```

```

52  if (iam.eq.nprocs-1) then
53      h(n+1) = hL
54  end if
55
56  xval = (iam*n) * dx
57  k(0) = a*xval**2 + b*xval + c
58  xval = ((iam+1)*n + 1) * dx
59  k(n+1) = a*xval**2 + b*xval + c
60
61  do i=1,n
62      h(i) = 0.d0
63
64      xval = (iam*n + i) * dx
65      k(i) = a*xval**2 + b*xval + c
66  enddo
67
68  !=====
69  !start iterations
70  !=====
71  do j=1,nit
72      ! red-black Gauss-Seidel (0=red,1=black)
73      ! with GS, don't need hnew, just updating h directly
74      !$omp parallel do
75      do m=0,1
76          do i = m,n,2
77              h(i)=(h(i-1)+h(i+1))/2.d0
78          enddo
79      end do
80
81      !reinitialize boundary conditions
82      if (iam.eq.0) h(0)=h0
83      if (iam.eq.nprocs-1) h(n+1)=hL
84
85      ! average leakage flux calculation (only do this one time)
86      ! approximate dh/dx using 2nd order central diff
87      !Q = 0
88      !do i = 1,n
89          !!dhdxdx = 0.5*(h(i+1) - h(i-1))
90          !dhdxdx = (h(i) - h(i-1))
91          !Q = Q + k(i) * qarea * dhdxdx * dx
92      !enddo
93  enddo
94  !=====
95  !end iterations
96  !=====
97
98  !=====
99  !end timer
100 !=====
101 call cpu_time(time2)
102 total_time=time2-time1
103
104 !=====
105 ! calculate error using analytical solution

```

```

106 !=====
107     errsq=0.d0
108     do i=1,n
109         !hanal=h0+(hL-h0)*(iam*n+i)/(size+1) ! version 0
110
111         xval = (iam*n + i) * dx
112         fanal = (2/sqrt(4*a*c-b*b))*atan((2*a*xval+b)/sqrt(4*a*c-b*b))
113         hanal = c0*fanal + c1
114
115         errsq = errsq + (h(i)-hanal)**2
116         !print *, "errsq = ",errsq
117     enddo
118
119 !=====
120 ! print results
121 !=====
122     !print *, "sumsqerr = ",sumsqerr
123     error=sqrt(errsq)/dble(size) ! can only do this on proc 0
124
125     !mflops = 2*dble(nit)*dble(size)*1e-6/(total_time) ! version 0
126     mflops = 8*dble(nit)*dble(size)*1e-6/(total_time) ! versions 1-5
127
128     print *, "nprocs =",nprocs
129     print *, "error =",error
130     print *, 'mflops =',mflops
131     print *, 'Total time =',total_time
132
133     !print *, 'Q =',Q
134 end

```

Fortran Code: Problem 9

```
1  ! Andrew Navratil
2  ! CE 791 Final Exam – Problem 9
3
4  program p9
5  implicit none
6  include "mpif.h"
7
8  ! === constants ===
9  real(8), parameter :: PI = 4.D0*DATAN(1.D0)
10
11  integer, parameter :: n = 1e6
12
13  ! === variables ===
14  integer i,j,k,iter
15  real(8) cr
16
17  character(len=50) :: filename
18
19  ! === MPI variables ===
20  real(8) time1,time2,total_time,ierr
21  integer nprocs,iam,minid
22
23  ! grid data displayed in pdf assumes y=100 is bottom row, y=400 is top row
24  ! to view it like x-y grid with origin in bottom left corner
25  real(8), dimension(5) :: xgrid
26  real(8), dimension(4) :: ygrid,tgrid
27  real(8), dimension(4,5,4) :: obsvals ! observation data: k,x,y
28  real(8) cs,val1,val2,x0,y0,m0,curc,minc
29  real(8), dimension(2) :: mincl,mincg,x0minl,y0minl,m0minl
30  real(8), allocatable :: x0min(:), y0min(:), m0min(:)
31
32  xgrid = (/ 100, 300, 500, 700, 900 /)
33  ygrid = (/ 100, 200, 300, 400 /)
34  tgrid = (/ 30,60,90,120 /)
35  obsvals(1,:,1) = (/0.0000 ,0.0000 ,0.0000 ,0.0000 ,0.0000 /)
36  obsvals(1,:,2) = (/0.0000 ,0.0000 ,0.0000 ,0.0000 ,0.0000 /)
37  obsvals(1,:,3) = (/0.0611 ,9.0783 ,1.6974 ,0.0004 ,0.0000 /)
38  obsvals(1,:,4) = (/0.0114 ,1.7028 ,0.3154 ,0.0001 ,0.0000 /)
39  obsvals(2,:,1) = (/0.0000 ,0.0000 ,0.0000 ,0.0000 ,0.0000 /)
40  obsvals(2,:,2) = (/0.0013 ,0.1883 ,0.9862 ,0.1882 ,0.0013 /)
41  obsvals(2,:,3) = (/0.0158 ,2.3119 ,11.9102 ,2.3336 ,0.0153 /)
42  obsvals(2,:,4) = (/0.0000 ,0.0067 ,0.0359 ,0.0068 ,0.0000 /)
43  obsvals(3,:,1) = (/0.0000 ,0.0046 ,0.0740 ,0.1290 ,0.0241 /)
44  obsvals(3,:,2) = (/0.0015 ,0.2217 ,3.5988 ,6.4052 ,1.2005 /)
45  obsvals(3,:,3) = (/0.0003 ,0.0427 ,0.6870 ,1.1862 ,0.2238 /)
46  obsvals(3,:,4) = (/0.0000 ,0.0000 ,0.0005 ,0.0009 ,0.0002 /)
47  obsvals(4,:,1) = (/0.0001 ,0.0125 ,0.3435 ,1.8402 ,1.8562 /)
48  obsvals(4,:,2) = (/0.0001 ,0.0189 ,0.5285 ,2.7696 ,2.8031 /)
49  obsvals(4,:,3) = (/0.0000 ,0.0004 ,0.0124 ,0.0651 ,0.0654 /)
50  obsvals(4,:,4) = (/0.0000 ,0.0000 ,0.0000 ,0.0000 ,0.0000 /)
51
```

```

52  !=====
53
54  ! initialize MPI
55  call MPI_INIT(ierr)
56  call MPI_COMM_RANK(MPI_COMM_WORLD, iam, ierr)
57  call MPI_COMM_SIZE(MPI_COMM_WORLD, nprocs, ierr)
58
59  allocate(x0min(0:nprocs-1),y0min(0:nprocs-1),m0min(0:nprocs-1))
60
61  ! start timer for mflops calculation
62  time1 = MPI_Wtime()
63
64  ! generate random numbers and distribute to procs cyclically
65  call random_seed()
66
67  ! calculate sums, distribute workload among all procs
68  minc = 1e15
69  do iter = iam+1,n,nprocs
70      call random_number(cr)
71      x0 = floor(cr*1000)
72      call random_number(cr)
73      y0 = floor(cr*500)
74      call random_number(cr)
75      m0 = floor(cr*100)
76      !write(*,*) x0,y0,m0
77
78      ! loop each time step and x,y location in observation data
79      do k = 1,4
80          do i = 1,5
81              do j = 1,4
82                  val1 = (xgrid(i)-x0-5*tgrid(k))**2 / (4*100*tgrid(k))
83                  val2 = (ygrid(j)-y0-2*tgrid(k))**2 / (4*10*tgrid(k))
84                  cs = (10e5*m0)/(8*(PI*tgrid(k))**(1.5)*sqrt(100.*10.))
85                  cs = cs*exp(-val1-val2-0.001*tgrid(k))
86  !print *, 'obs=', obsvals(k,i,j), ' cs=', cs
87                  curc = curc + (obsvals(k,i,j) - cs)**2
88              end do
89          end do
90      end do
91
92      !print *, minc
93      if (curc.lt.minc) then
94          x0min1 = x0
95          y0min1 = y0
96          m0min1 = m0
97          minc = curc
98          !print *,minc,x0,iam
99      end if
100
101      mincl(1) = minc
102      mincl(2) = dble(iam)
103  end do
104
105  call MPI_ALLREDUCE(mincl,mincg,1,MPI_2DOUBLE_PRECISION,&

```

```

106     MPI_MINLOC,MPI_COMM_WORLD,ierr)
107
108     call MPI_GATHER(x0minl,1,MPI_DOUBLE_PRECISION, &
109         x0min,1,MPI_DOUBLE_PRECISION,0,MPI_COMM_WORLD,ierr)
110     call MPI_GATHER(y0minl,1,MPI_DOUBLE_PRECISION, &
111         y0min,1,MPI_DOUBLE_PRECISION,0,MPI_COMM_WORLD,ierr)
112     call MPI_GATHER(m0minl,1,MPI_DOUBLE_PRECISION, &
113         m0min,1,MPI_DOUBLE_PRECISION,0,MPI_COMM_WORLD,ierr)
114
115     ! end timer
116     time2 = MPI_Wtime()
117     total_time = time2-time1
118
119     ! print data and execution time to final.out file
120     if (iam.eq.0) then
121         !do i=0,nprocs-1
122         !print *, x0min(i),y0min(i),m0min(i)
123         !end do
124         minid = mincg(2)
125         print *, 'minid=',minid
126         print *, 'min x0=',x0min(minid),' min y0=',y0min(minid),' min m0=',m0min(minid)
127         print *, 'nprocs =',nprocs
128         print *, 'total time =',total_time
129     end if
130
131     ! finalize MPI
132     call MPI_FINALIZE(ierr)
133
134 end program p9

```

Matlab Code

```
1 close all; clear all; clc;
2
3 % benchmarking code executed on bezier
4 % mflops, exec time, num procs printed to final.out and copied in here
5
6 %procs = [1 2 4 8 16] % p3
7 %mflops = [670 1293 2265 2790 2653] % p3
8 %times = [1.792e-2 9.282e-3 5.297e-3 4.301e-3 4.522e-3] % p3
9
10 %procs = [1 2 4 8] % p4
11 %mflops = [3.331 0.2727 .1604 .113] % p4
12 %times = [3.002e-7 7.332e-6 2.494e-5 7.070e-5] % p4
13
14 % errors: v2: 1.01e-2, gs: 1.96e-2, gs openmp: 1.96e-2
15 %procs = [1 2 4 8 16] % p7
16 %mflopsv2 = [815 390 329 180 73.9]
17 %timesv2 = [.491 1.03 1.22 2.22 5.41]
18 %mflopsgs = [3193 482 361 181 78.5]
19 %timesgs = [.125 .830 1.11 2.20 5.09]
20 %mflopsopenmp = [11067 5355 3047 1355 691]
21 %timesopenmp = [3.61e-2 7.47e-2 .131 .295 .578]
22
23 procs = [1 2 4 8 16] % p9
24 times = [3.00 1.66 1.03 1.01 .990]
25
26 plot(procs,times,'*k—','MarkerSize',10);
27 %plot(procs,timesv2,'*--','MarkerSize',10,'DisplayName','v2');
28 %hold on;
29 %plot(procs,timesgs,'+--','MarkerSize',10,'DisplayName','RBGS');
30 %plot(procs,timesopenmp,'^--','MarkerSize',10,'DisplayName','OpenMP');
31 xlabel('Number of Processors');
32 ylabel('Elapsed Time (s)');
33 set(gca,'FontSize',16);
34 %legend('location','northwest');
35
36 %figure;
37 %plot(procs,mflops,'*b—','MarkerSize',10);
38 %plot(procs,mflopsv2,'*--','MarkerSize',10,'DisplayName','v2');
39 %hold on;
40 %plot(procs,mflopsgs,'+--','MarkerSize',10,'DisplayName','RBGS');
41 %plot(procs,mflopsopenmp,'^--','MarkerSize',10,'DisplayName','OpenMP');
42 %xlabel('Number of Processors');
43 %ylabel('Mflops');
44 %set(gca,'FontSize',16);
45 %legend('location','northeast');
```